Final Project Report

1. Introduction

Background

A van rental business has to keep track of its vans, its customers, who rented what, when the van is due back, and which vans are in the shop. When this is done on paper, mistakes are easy to make. Staff may not know if a van is free. They may forget a due date. They may add a late fee the wrong way.

The Passenger Van Rental System is a Java program that keeps all of this in one place. It is a console program. The staff pick an option from a menu and type the details. The data is saved in text files, so it is still there the next time the program is opened.

Problem Statement

The rental business has these problems:

1. It is hard to know which vans are available, rented, or under maintenance.
2. Customer details and ID information must be kept the same way every time.
3. Rental dates, returns, and late fees must be tracked without calculation mistakes.
4. A van under maintenance must not be rented.
5. Staff need a way to find due and overdue rentals and remind the customers.
6. Records must not be lost when the program closes.

Objectives

The general objective is to build a Java program that manages vans, customers, rentals, staff, and maintenance records, and saves them in files, using object-oriented programming.

The specific objectives are:

1. Add, view, update, and delete customer records.
2. Add, delete, view, and search vans (Standard and Luxury).
3. Record rentals and returns, and work out the rental fee and the late fee.
4. Keep rental records that show the customer, van, dates, status, and staff.
5. Record maintenance and stop a van under maintenance from being rented.
6. Show due and overdue rentals, and send email receipts and SMS reminders (simulated).
7. Use encapsulation, inheritance, polymorphism, abstraction, object relationships, collections, exception handling, and file persistence.
8. Check user input and save and load records from text files.

Target Users

User	How they use the system
Rental Staff	Main users. They manage customers and vans, rent and return vans, and record maintenance. They type their staff ID for actions that need it. There is no login.
Customers	Their details and rentals are recorded by staff. They do not use the program. They can get a receipt or SMS (simulated).
Owner or Supervisor	Can read the lists and records to check van status, rentals, overdue items, and maintenance. There is no separate supervisor account.

Scope and Limitations

Scope

- Manages Standard and Luxury vans with plate number, model, seats, and status.
- Customer add, view, update, and delete. A customer with a rental record cannot be deleted.
- Rentals, returns, late fees, due and overdue checks, and maintenance records.
- Email receipts and SMS reminders are shown on screen and saved in a log file.
- All data is saved in local text files.

Limitations

- It is a console program. There is no screen with buttons, no website, and no phone app.
- It uses text files, not a database.
- Email and SMS are not real. Nothing is sent to a real service.
- There is no login or password. Staff IDs are only checked for some actions.
- It is made for one user at a time. It does not work across branches.
- There is no online booking, online payment, or GPS.

2. Analysis

Comparing the Programming Paradigms

	Procedural	Functional	Object-oriented
Main unit	Procedures that work on data	Functions that take values and return new values	Objects that hold data and behavior together
Data	Often changed by procedures	Prefers data that does not change	Kept inside each object and changed through its methods
Good for	Small tasks	Checking input, counting, and tasks that are easy to test	Programs with real things that keep changing
Weak point	Many procedures depend on the same data	Hard to use for things that change, like van status	Too much for a very small program

If we used a procedural style, every procedure would need to agree on what each flag means. The same checks would be copied into rent, return, and maintenance. Over time they may not match.

If we used a functional style, a late fee function would be easy to test. But reading files, typing input, and changing van status would still have to be done outside the functions.

Why We Chose OOP

The rental business is made of real things: vans, customers, staff, rentals, and maintenance jobs. Each one has its own data and its own actions. Each became a class.

- A Standard van and a Luxury van have different prices. Each van works out its own fee.
- Customers and staff share a name, contact number, and email. They share one parent class.
- Email and SMS are different ways to send a message, but the system uses them the same way.
- Status changes (rented, returned, under maintenance) are kept inside the objects.

We still use some procedural code in the menu loop of Main. The ideas of the other paradigms can still be used inside an OOP program.

3. Design

The full diagrams are in the Phase 3 document (System and UML). This section gives a short summary of them.

Use Case Diagram

There is one user, the Rental Staff. They can use all 16 menu options and the exit option:

Group	**Use cases**
Customers	[1] Add, [2] Delete, [3] Update, [8] View all
Vans	[4] Add, [5] Delete, [6] View by status, [7] Search by plate
Rentals	[9] Rent a van, [10] Return a van, [11] View rentals, [12] Send expiration SMS, [13] View overdue
Maintenance	[14] Send van to maintenance, [15] Finish maintenance, [16] View history
Exit	[0] Save and exit

The use cases include these smaller steps: Validate Input, Verify Rental Staff ID, Send Email Receipt, Send SMS Reminder, and Save Data to Files.

Class Diagram

Class	Type	What it does
Main	Class	The menu and all typing and printing. Reads and checks input. Catches RentalException.
RentalSystem	Class	Keeps the lists. Has the rules for renting, returning, and maintenance.
FileManager	Class	Saves and loads the five text files.
Person	Abstract class	Name, contact number, email. Parent of Customer and Employee.
Customer	Class	Adds customer ID, address, ID type, and ID number.
Employee	Class	Adds the staff ID.
Van	Abstract class	Plate, model, seats, available, under maintenance.
StandardVan	Class	Rate P3000 per day.
LuxuryVan	Class	Rate P4500 per day plus P500 cleaning fee.
Rental	Class	Links a customer, a van, and the staff. Has dates, fees, and return details.
MaintenanceRecord	Class	Links a van and a staff member. Has a note and dates.
Notification	Interface	One method: send(recipient, message).
EmailNotification	Class	Shows an email and writes it to the log.
SmsNotification	Class	Shows an SMS and writes it to the log.
RentalException	Class	Error for broken rules. Extends Exception.

Relationships

- Inheritance: Customer and Employee extend Person. StandardVan and LuxuryVan extend Van. RentalException extends Exception.
- Interface: EmailNotification and SmsNotification implement Notification.
- Association: Rental holds a Customer, a Van, and an Employee (rentedBy and returnedBy). MaintenanceRecord holds a Van and an Employee.
- Aggregation: RentalSystem holds lists of Van, Customer, Rental, Employee, and MaintenanceRecord (0 or more of each).
- Composition: RentalSystem creates its own FileManager and its two Notification objects.
- Dependency: RentalSystem throws RentalException.

Activity Diagrams

The overall activity is the menu loop. The program loads the files, shows the menu, reads a choice, does the action, and saves all data. If a rule is broken, the error is shown and the menu comes back. It repeats until the user picks 0.

The rent a van activity has these checks, in order: staff ID exists, customer ID exists, a van is available, the van exists and is free, the customer has an address and a valid ID, and days is 1 or more. If any check fails, an error is shown and no rental is made. If all pass, the rental is made, the van is set to Rented, and the email receipt is shown.

Sequence Diagrams

- **Rent a van:** Main asks RentalSystem to find the staff and customer, list rentable vans, and check the van. Then rentVan makes a Rental. The Rental asks the Van for its fee. The van is set to not available, the rental is added to the list, and the receipt goes through EmailNotification.
- **Return a van:** RentalSystem finds the rental and marks it returned. The Rental works out the late days and the late fee (P250 for each day after the end date). The van becomes available again. Main prints the late fee and total.
- **Load and save:** At start, RentalSystem asks FileManager to load the five files. After every menu action, it asks FileManager to save them.

4. Implementation

Tools and How to Run

- Language: Java.
- Interface: console menu.
- Storage: text files.
- Compile: `javac -d bin src/*.java`
- Run (from the project folder that has the data files): `java -cp bin Main`

The program has 15 classes for the application and 1 test class (TestRental). The application code is about 1,200 lines.

Project Files

File	Use
vans.txt	One van per line, for example Standard,NEW111,Toyota Hiace,15,true,false
customers.txt	One customer per line
employees.txt	Rental staff (E1, E2, E3)
rentals.txt	All rentals, including returned ones
maintenance.txt	Maintenance records
notifications_log.txt	Email and SMS messages that were sent (simulated)

The start data has 25 vans, 15 customers, and 3 staff. There are no rentals or maintenance records at the start.

Where Each OOP Concept Is Used

Concept	Where it is in the code	Why
Classes and objects	15 classes, such as Customer, Van, and Rental. Objects are made with new, for example new Customer(...) in addCustomer.	Each real thing is one class.
Encapsulation	Fields in Person, Van, Rental, and others are private. Other code uses getters and setters.	Data is changed only in controlled ways.
Inheritance	Customer and Employee extend Person. StandardVan and LuxuryVan extend Van.	Shared data is written once.
Polymorphism	Rental calls van.calculateFee(days) on a Van. The Standard or Luxury version runs. Notification objects send email or SMS the same way.	One call, different results.
Abstraction	Person and Van are abstract. Notification is an interface.	The parent says what must be done. The child says how.
Object relationships	Rental holds a Customer, a Van, and an Employee. MaintenanceRecord holds a Van and an Employee.	Records are linked to the real objects.
Collections	RentalSystem uses an ArrayList for vans, customers, rentals, employees, and maintenance records.	Many objects can be stored and searched.
Exception handling	RentalSystem throws RentalException. Main catches it and prints Error: message. Input methods handle wrong numbers.	A broken rule shows a message and does not crash the program.
File persistence	FileManager.saveAll writes the five files. The load methods read them back.	Data stays after the program closes.

Main Functions

- **Customers:** add, update, delete, and view. The ID number is shown with only the last 4 characters. When a customer is deleted, the free ID can be used again.
- **Vans:** add (type S or L), delete, search by plate (not case sensitive), and view in three groups: Available, Rented, and Under Maintenance.
- **Rent:** needs a staff ID, a customer with an address and valid ID, and a free van. The fee is worked out by the van type. An email receipt is shown and saved in the log.
- **Return:** needs a staff ID and a rental ID. If the van is late, the late fee is P250 for each late day. The van becomes available again.
- **Overdue and SMS:** lists rentals that are due today or late, and sends SMS reminders (simulated).

- **Maintenance:** a van that is rented cannot be sent to maintenance. A van under maintenance cannot be rented or deleted. Finishing maintenance makes the van available again.

Input Checks

Input	Rule
Any text	Cannot be empty and cannot have a comma (a comma would break the data file).
Contact number	11 digits, starting with 09.
Email	Must have a valid email format.
Address	Must be a complete address (not one word).
ID number	5 to 20 letters, digits, or dashes.
Numbers	Must be a whole number. The program asks again if it is not.
Van type	S or L.
Seats	At least 1.
Days	At least 1.

Fees

Fee	Amount
Standard van	P3000 x days
Luxury van	P4500 x days + P500 cleaning fee
Late fee	P250 x days after the end date

Example: a Luxury van for 2 days costs P4500 x 2 + P500 = P9500.

5. Testing and Results

The full test cases are in the Phase 5 document (Test and Debugging). The tests were run on the real program with a script (TestRental.java). Every test started with a fresh copy of the data files.

Test Summary

Group	Test cases	Passed	Failed
Customer (TC-01 to TC-18)	18	18	0
Van (TC-19 to TC-29)	11	11	0
Rental (TC-30 to TC-43)	14	13	1
Return (TC-44 to TC-49)	6	6	0
Overdue and SMS (TC-50 to TC-54)	5	5	0
Maintenance (TC-55 to TC-64)	10	10	0
Menu (TC-65 to TC-67)	3	3	0

Group	Test cases	Passed	Failed
File persistence (TC-68 to TC-73)	6	4	2
Total	**73**	**70**	**3**

70 of 73 tests passed (96%).

Failed Tests and Debugging

Bug	Test	What happens	Cause	Planned fix	Status
BUG-01	TC-71	The program crashes at start when vans.txt has a blank line. No menu is shown.	FileManager.loadVans reads p[1] on a blank line and throws ArrayIndexOutOfBoundsException. Only IOException and NumberFormatException are caught.	Skip blank lines and short lines. Also catch RuntimeException, like the other load methods.	Open
BUG-02	TC-73	After a van with a returned rental is deleted, the rental history is lost on the next start.	deleteVan only checks if the van is rented or under maintenance. loadRentals skips any rental whose van is missing.	Refuse to delete a van that has any rental, the same way deleteCustomer works.	Open
BUG-03	TC-43	A rental of 2147483647 days is accepted. The fee is about P6.4 trillion.	rentVan only checks that days is 1 or more.	Set a limit of 30 days and throw RentalException if it is more.	Open

The fixes are planned. They are not in the code yet. After they are added, the same tests must be run again and the new results written down.

6. Conclusion and Recommendations

Conclusion

The Passenger Van Rental System does what it was made to do. Staff can manage customers and vans, rent and return vans, see overdue rentals, record maintenance, and keep all data between runs. Wrong input shows a clear message in almost every test. 70 of 73 tests passed.

The project also shows the OOP ideas the course asks for. Standard and Luxury vans give different fees through the same method. Email and SMS share one interface. A custom exception reports broken rules. Data is saved to and loaded from text files.

We found three problems. They are listed in section 5.2. They do not affect normal use, but BUG-02 can lose rental history, so it should be fixed first.

Recommendations

1. Fix BUG-02, then BUG-01, then BUG-03. Run the 73 tests again after each fix.
2. Make the getters getVans(), getCustomers(), and getRentals() return copies of the lists, so other code cannot change them by mistake.
3. Make rental and maintenance IDs from a counter that is saved, not from the list size. IDs from the list size can repeat if records are deleted.
4. Move the rates (P3000, P4500, P500, P250) to one place, such as a settings file, so they are not fixed numbers in the code.
5. Add a login or PIN for staff.
6. Move the email and SMS logging code into one shared place. Both classes have the same code now.
7. Use a database in the future if the business grows or has more than one branch.

UML Diagram

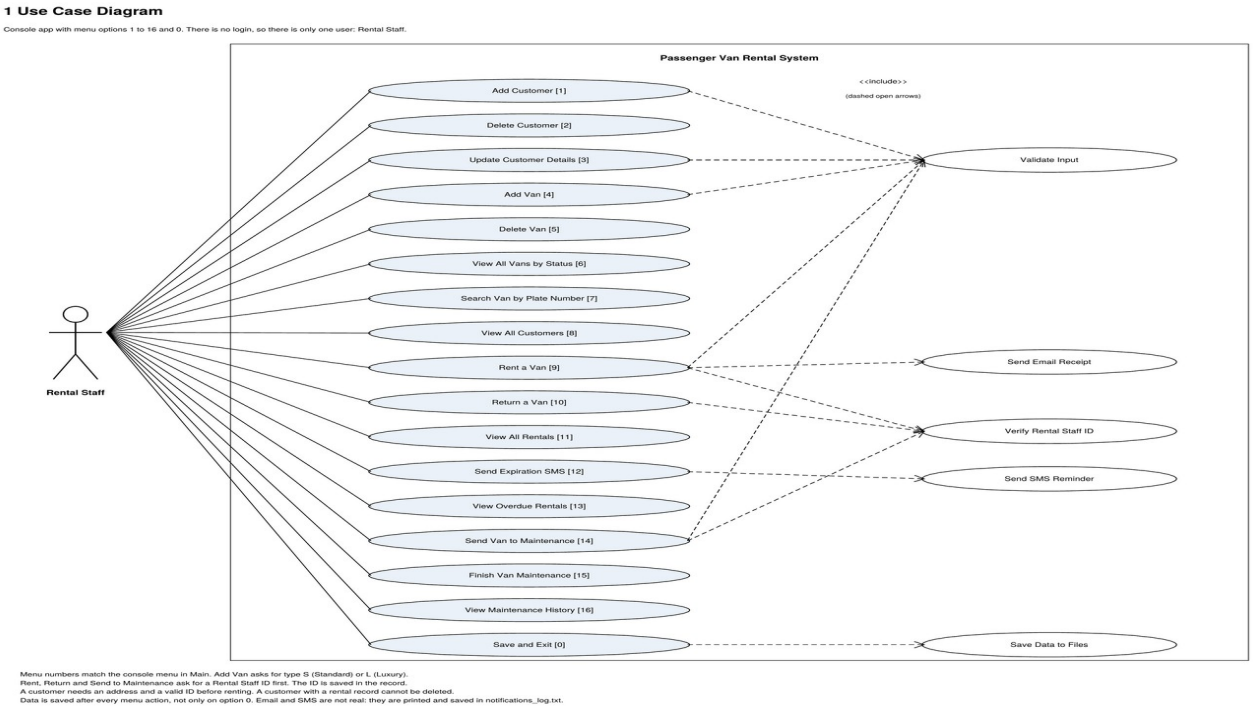


Figure 1. Use-Case Diagram

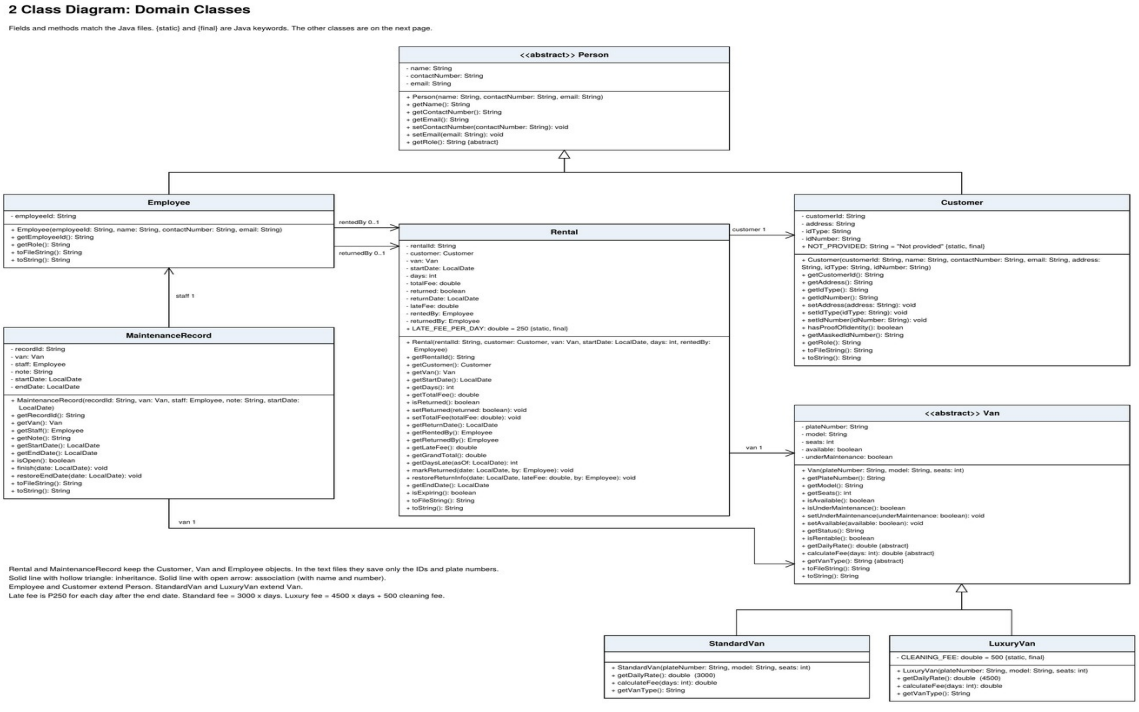


Figure 2. Domain Classes Diagram

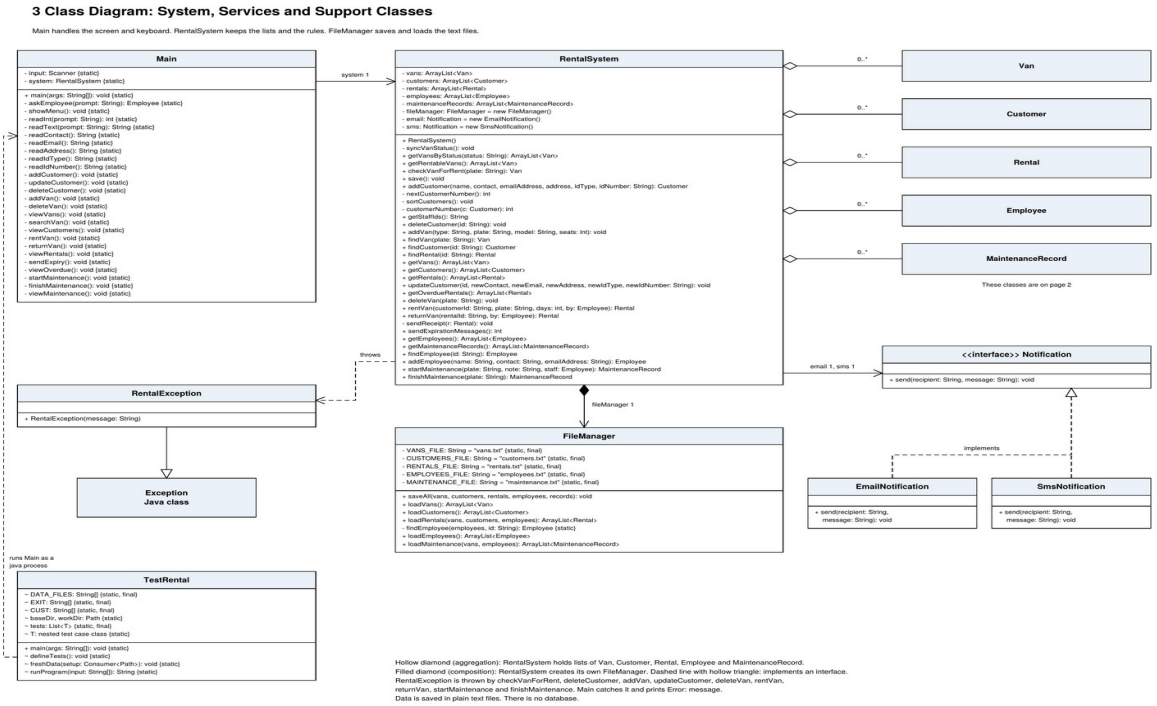


Figure 3. System & Support Diagram

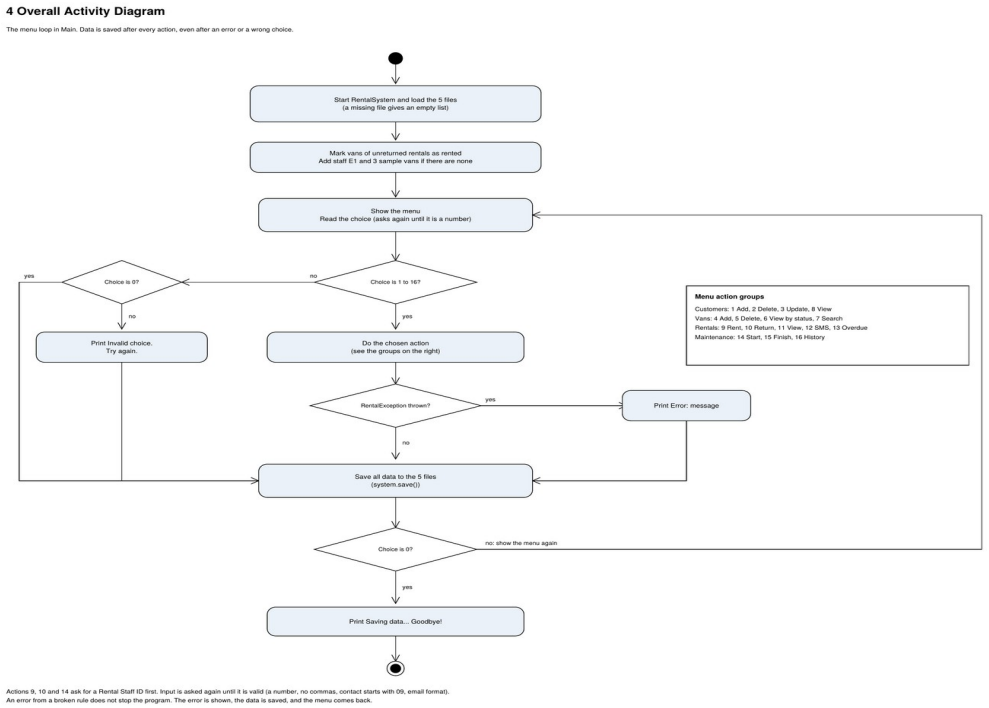


Figure 4. Overall Activity Diagram

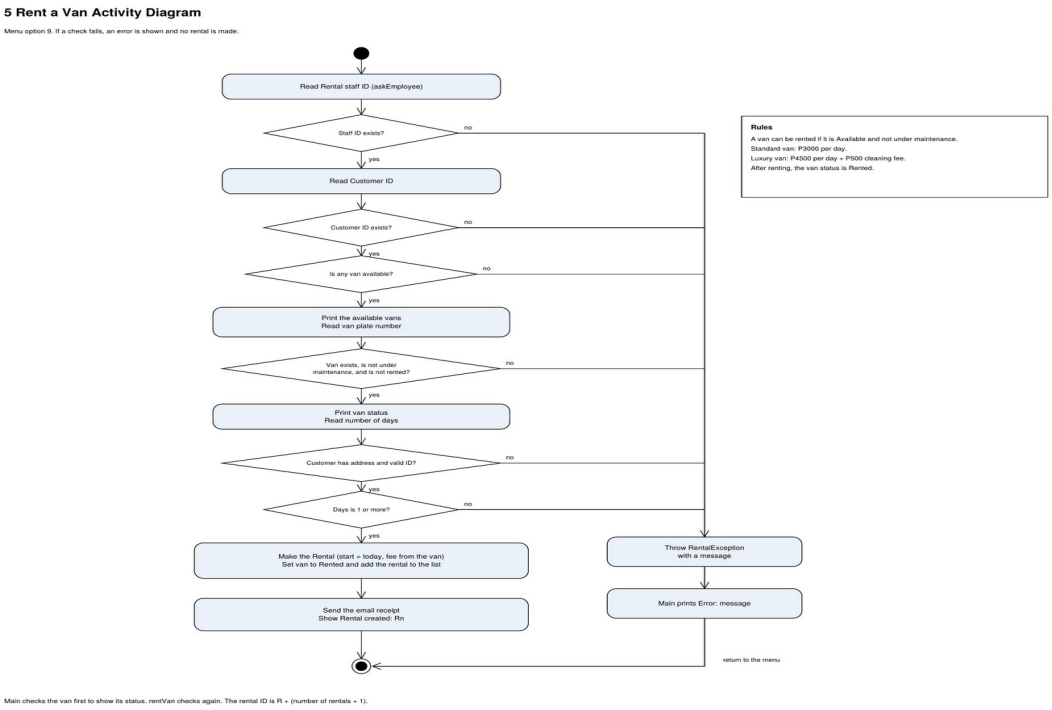


Figure 5. Rent-A-Van Acitvity Diagram

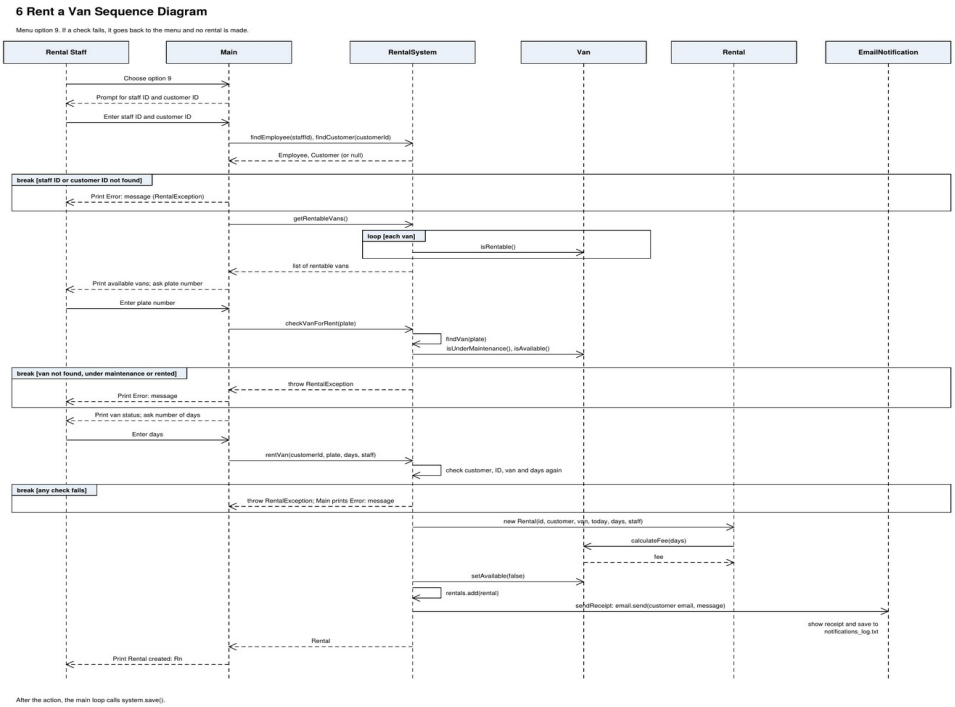


Figure 6. Rent-A-Van Sequence Diagram

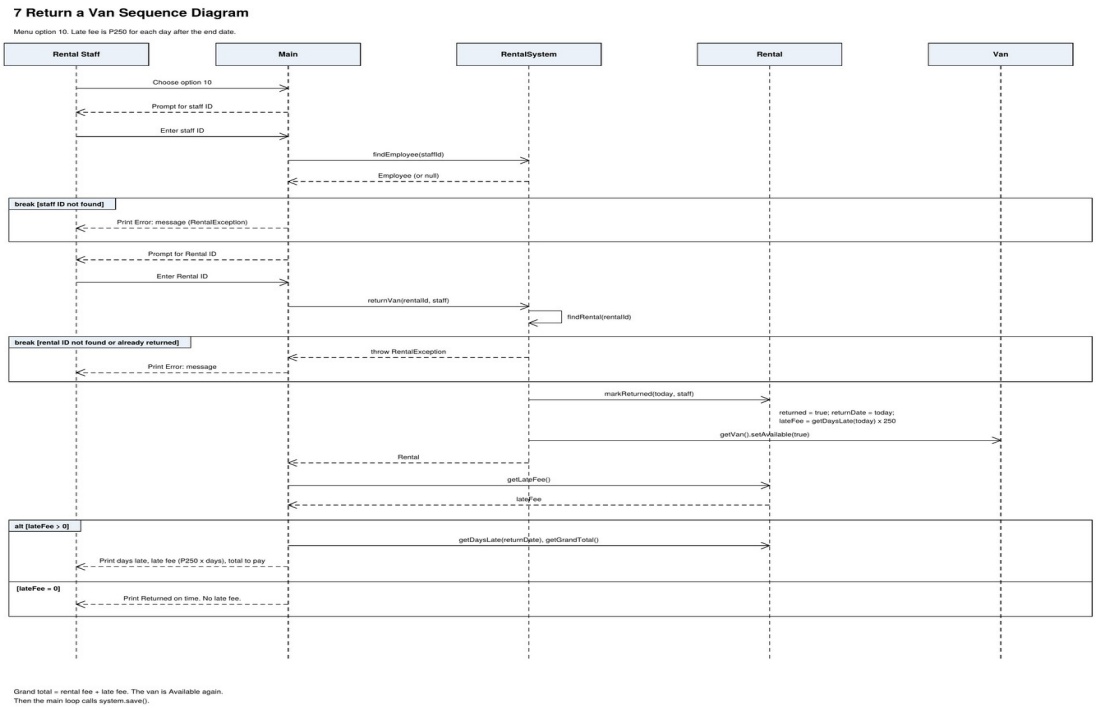


Figure 7. Return-A-Van Sequence Diagram

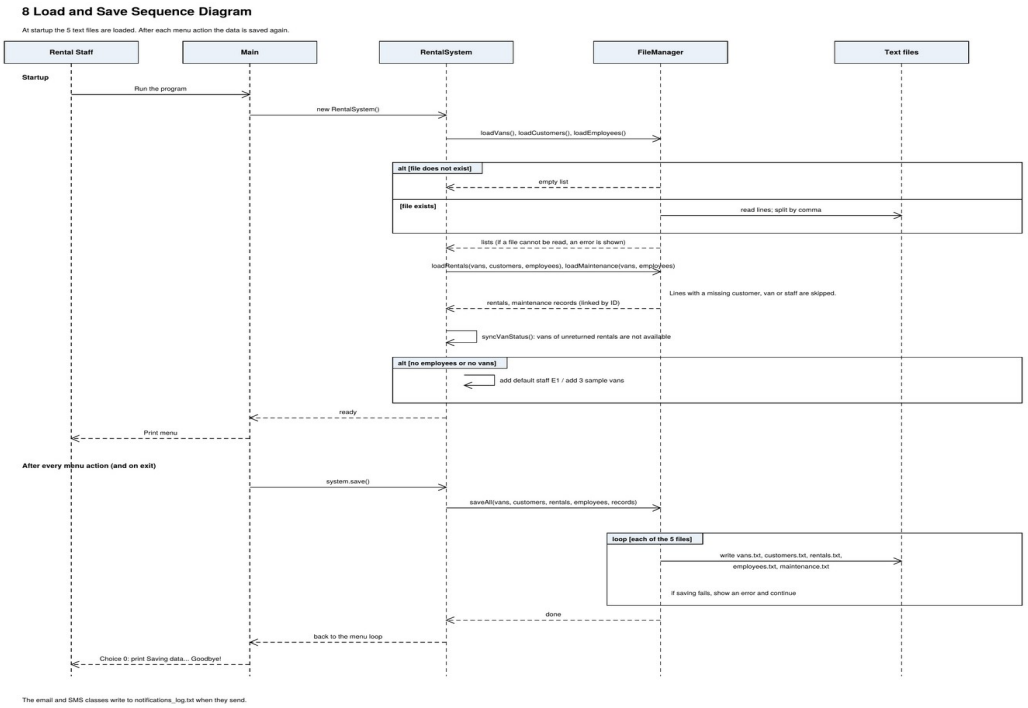


Figure 8. Load-Save Sequence Diagram